

Introspectable, Updatable, and Uncertainty-aware Classification of Language Model Instruction-following

Allen Schmaltz
Reexpress AI
USA
allen@re.express

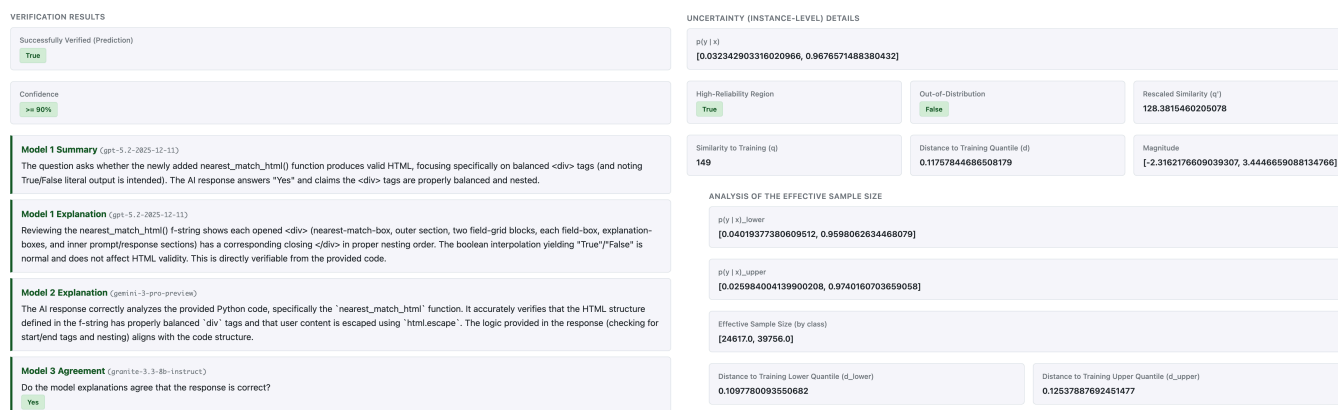


Figure 1: Uncertainty estimates over classification of language model instruction-following via the REEXPRESS MCP SERVER.

Abstract

In this system demonstration paper, we introduce an open-source implementation for training and testing Similarity-Distance-Magnitude (SDM) estimators for the task of binary classification of instruction-following of *closed-weight* language models (LMs). This SDM estimator provides an approximately conditional estimate of the predictive uncertainty over instruction-following, conditional on multiple closed-weight LMs and the representation space of an open-weight model. While it would be more robust to use as input to the SDM estimator the hidden-states of the underlying models, this indirect, compositional proxy is more reliable than verbalized uncertainty and adds a means of auditing the predictions against data with known labels. We release the code as an MCP Server to simplify adding interpretability-by-exemplar and locally updatable, uncertainty-aware instruction-following to agent-based pipelines. We further release OpenVerification1, a balanced set of over two million examples of instruction-following and associated rationales from recent closed-weight LMs, for bootstrapping domain-specific estimators. Finally, we discuss limitations of estimating the predictive uncertainty without access to the hidden-states of the tool-calling LM and provide practical guidance for applications.

CCS Concepts

• Computing methodologies → Machine learning; Natural language processing; Uncertainty quantification.

Keywords

Approximately conditional calibration, Interpretability-by-exemplar, Classification of instruction-following, Model ensembles

ACM Reference Format:

Allen Schmaltz. 2026. Introspectable, Updatable, and Uncertainty-aware Classification of Language Model Instruction-following. In *ACM Conference on AI and Agentic Systems (CAIS '26)*, May 26–29, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3786335.3813214>

1 Introduction

Reliability remains a central challenge for production-level deployment of agent-based systems constructed from closed-weight neural network language models (LMs) [29]. The underlying technical challenge is of a foundational modeling nature; as often deployed, neural network models with non-identifiable parameters can be grossly mis-calibrated for a given domain-specific agent-based task, without any demonstrative warning to the user. This is a consequence of the output logits (or downstream transforms thereof, such as the verbalized uncertainty) being inadequate signals of the epistemic (reducible) uncertainty, which precludes simple post-hoc calibration methods, such as temperature scaling [16, 30], for agent settings subject to covariate and labels shifts, which are not uncommon for systems operating over high-dimensional inputs. As an additional challenge, conditional branching decisions in agent settings require estimates of the predictive uncertainty for selective classification [5, 10, inter alia], but class- and prediction-conditional calibration



This work is licensed under a Creative Commons Attribution 4.0 International License. CAIS '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2415-2/2026/05
<https://doi.org/10.1145/3786335.3813214>

in high-probability regions is not a quantity typically targeted by existing classes of calibration methods [1, 4, 6, 9, 21, 22, 28, 35, 36, inter alia]. In short, contemporary closed-weight LMs available via developer APIs “don’t know what they do[n’t] know”, and this limitation renders multi-stage LM-agent-based pipelines over such models inherently brittle.

With access to the hidden-states of the underlying model, the modeling challenges are greatly simplified, with principled statistical approaches available to robustly calibrate the predictive uncertainty based on the effective sample size of a held-out calibration set. Concretely, we can calibrate the model via a SIMILARITY-DISTANCE-MAGNITUDE (SDM) estimator [32], which models the predictive uncertainty based on constraints on the feature-representation space of the model over the training set of the estimator. As a result of the dense matching into the training set, SDM estimators also provide interpretability-by-exemplar, or *instance-wise* data valuation, for auditing the prediction against the instances with known labels.

In the present work, we consider the related, but intrinsically harder (Appendix A), inferential task of calibrating and interpreting the model without access to its hidden-states, which is a common setting with contemporary API-based language models. We frame the task as *binary classification of instruction-following* of the tool-calling LM, taking as input to an SDM estimator an ensemble of multiple closed-weight LMs and the hidden-states of an open-weight model. The resulting estimates can be used for conditional-branching in multi-stage LM-agent-based pipelines, and we provide a Model Context Protocol (MCP) Server [2] that simplifies adding these capabilities to existing systems.

In summary, in this system demonstration:

- We introduce an approach for constructing SDM estimators over binary classification of instruction-following of a closed-weight LM via an ensemble of multiple closed-weight LMs and the hidden-states of an open-weight model. This is more reliable than using the verbalized uncertainty to achieve class- and prediction-conditional accuracy at least α , a value near 1, as needed for selective classification and branching, and provides a principled means of auditing the predictions against the data with known labels.
- We introduce the open-source REEXPRESS MCP SERVER¹, which simplifies adding these uncertainty quantification and interpretability capabilities to LMs that support the MCP protocol. We release a trained SDM estimator as a demonstration of these capabilities for developers constructing domain-specific estimators, and as a pre-trained model for low-risk programming and data science settings.
- We introduce OPENVERIFICATION1², a dataset of over two million examples of instruction-following and associated rationales from recent closed-weight LMs, for bootstrapping domain-specific estimators.
- We discuss limitations of estimating the predictive uncertainty without access to the hidden-states of the tool-calling LM and provide practical guidance for applications.

¹https://github.com/ReexpressAI/reexpress_mcp_server

²<https://huggingface.co/datasets/ReexpressAI/OpenVerification1>

2 Task setting: Binary classification of instruction-following

We model the predictive uncertainty of the tool-calling model, $\mathcal{M}_t$, for which we do not have access to the hidden-states nor training data, as binary classification of instruction-following. $\mathcal{M}_t$ will be used to generate a response (RESPONSE) to the user’s instructions (QUERY), and $\mathcal{M}_t$ will seek, via an external tool-call, an estimate of whether its response fulfilled the user’s instructions. The semantics of that classification are determined by the particular prompts (Appendix D) and calibration data (§ 4) used, but at a high-level we seek to *classify whether or not the generated response from $\mathcal{M}_t$ correctly and faithfully answers the user’s instructions*, where the ground-truth labels correspond to $y \in \{0 = \text{Not Verified}, 1 = \text{Verified}\}$. In addition to the prediction, $\hat{y}$, we seek an estimate of $p(y | x)$, the predictive uncertainty of that classification decision via an estimator, $\mathcal{E}$, such as an SDM estimator or the verbalized uncertainty, the latter of which is a real-valued probability in $[0, 1]$ expressed via the output tokens.

To construct the function called by $\mathcal{M}_t$, we have access to two additional closed-weight LMs and one open-weight LM, all of which are treated as conditioned (i.e., prompted) [33] classifiers [34]. $\mathcal{M}_1$, a closed-weight LM, takes as input a verification prompt that cross-encodes QUERY and RESPONSE (from $\mathcal{M}_t$)³, and generates SUMMARY₁, a short written summary of QUERY and RESPONSE; $\hat{y}_1$, a binary estimate of the verification prompt; $\hat{p}(y_1 | x_1)$ via the verbalized uncertainty; and RATIONALE₁, a brief written explanation for $\hat{y}_1$ and $\hat{p}(y_1 | x_1)$. $\mathcal{M}_2$, another closed-weight LM, takes the same input as $\mathcal{M}_1$, and similarly (but without a summary) generates $\hat{y}_2$; $\hat{p}(y_2 | x_2)$ via the verbalized uncertainty; and RATIONALE₂. Finally, $\mathcal{M}_a$, an open-weight model, takes as input a binary agreement prompt that cross-encodes SUMMARY₁, RATIONALE₁, and RATIONALE₂, and generates a single Yes or No token, returning $\hat{y}_a$, a binary estimate of the agreement prompt; $\mathbf{h}_a$, a concatenation of the mean-pooled final-layer hidden-states across sequence positions with the hidden-state of the final token position corresponding to $\hat{y}_a$; and $\hat{p}(y_a | x_a)$ via the output logits of the final token position.⁴

We assume labeled samples drawn from the reference class against which we seek to calibrate. This consists of a training dataset, $\mathcal{D}_{tr} = \{(\text{QUERY}_n, \text{RESPONSE}_n, y_n)\}_{n=1}^N$ of inputs paired with their corresponding ground-truth discrete labels, $y \in \mathcal{Y} = \{0, 1\}$, and a labeled calibration dataset, $\mathcal{D}_{ca}$, drawn from the same distribution as $\mathcal{D}_{tr}$.

3 Methods

3.1 SDM Activation and Estimator

SDM estimators are predicated on the SDM activation, which is a more robust and interpretable formulation of the commonly-used softmax, here specifically for binary classification:

$$\text{SDM}(z')_i = \frac{(2 + q)^{d \cdot z'_i}}{\sum_{c=0}^1 (2 + q)^{d \cdot z'_c}}, i \in \{0, 1\} \quad (1)$$

³In practice, both $\mathcal{M}_1$ and $\mathcal{M}_2$ can optionally access QUERY directly without mediation through $\mathcal{M}_t$, which is useful when the context (e.g., attached code) is large.

⁴This basic setup can be readily extended to additional models. In the present work, $\mathcal{M}_2$ is not tasked with generating a summary to save token costs and to minimize the length of the input to $\mathcal{M}_a$. Additionally, $\mathcal{M}_a$ operates over SUMMARY₁, rather than the original QUERY and RESPONSE, to further minimize the context length.

where $q \in [0, |\mathcal{D}_{\text{tr}}|] \cap \mathbb{Z}$ (SIMILARITY) is the count of consecutive (depth-wise) nearest matches (by L^2 distance) in $\mathcal{D}_{\text{tr}}$ that are correctly predicted *and* match $\hat{y}$ of the test instance. DISTANCE, $d \in [0, 1]$, is the minimum (across classes) 1-quantile of the L^2 distance to the nearest match (d_{nearest}) in $\mathcal{D}_{\text{tr}}$ relative to the distribution of such distances observed in $\mathcal{D}_{\text{ca}}$, where the empirical CDFs (eCDFs) are calculated class-wise to guard against class-wise skew in the distribution of outliers:

$$d = \min \left(1 - \text{eCDF}_{\text{ca}}^{y=0}(d_{\text{nearest}}), 1 - \text{eCDF}_{\text{ca}}^{y=1}(d_{\text{nearest}}) \right) \quad (2)$$

The MAGNITUDE, $z' \in (-\infty, \infty)$, is the typical distance to the decision boundary encoded in the output logits, but specifically from the output of an exemplar adaptor. This adaptor is a 1-D CNN ($g : \mathbf{h} \in \mathbb{R}^D \mapsto \mathbf{h}' \in \mathbb{R}^M$), the filter applications of which form $\mathbf{h}'$, the distilled representation of the network used for the dense matching to calculate q and d , followed by a linear output layer ($\mathbf{h}' \in \mathbb{R}^M \mapsto \mathbb{R}^2$).

Despite its simplicity, the SDM activation provides a substantively more robust substrate than the standard softmax for estimating downstream quantities of interest, because endogenous modeling of q and d provides more reliable signals of the epistemic uncertainty than the highly lossy compression of epistemic uncertainty contained in z' alone. In order to calibrate selective classification quantities of interest given these signals of the epistemic uncertainty, we seek to partition the output distribution via thresholds on each of q , d , and $\text{SDM}(z')$ via a held-out set to determine the well-calibrated, high-probability region. One approach is to manually set thresholds⁵; the SDM *estimator* instead seeks to partition the distribution in a fully data-driven manner, with the user simply choosing the calibration data and the probability threshold α . To do so, q is rescaled to take into account d and $\text{SDM}(z')$, given $\hat{y}$:

$$q' = \min \left(q, (2 + q)^{\text{SDM}(z')_{\hat{y}}} \right) \quad (3)$$

Next, the iterative algorithm of [32] searches for the value $q'_{\min}$ and class-wise output thresholds $[\psi_0, \psi_1]$ such that Eq. 4 partitions the distribution of $\mathcal{D}_{\text{ca}}$ into a HIGH-RELIABILITY (SDM_{HR}) region for which the class- and prediction-conditional accuracy is at least α :

$$\text{SDM}_{\text{HR}} := \begin{cases} \hat{y} & \text{if } q' \geq q'_{\min} \wedge \text{SDM}(z')_{\hat{y}} \geq \psi_{\hat{y}} \\ \perp & \text{otherwise} \end{cases} \quad (4)$$

where $\perp$ indicates a rejected (non-admitted) point and $\hat{y} = \arg \max z'$.

Finally, to account for noise in the eCDFs of Eq. 2 given the effective sample size, which is conservatively assumed to be increasing in q' class-wise over $\mathcal{D}_{\text{ca}}$, the DKW inequality [8] is used to construct a band around the prediction-conditional estimate $\text{SDM}(z')_{\hat{y}}$ by recalculating Eq. 1 with the lower and upper estimates of the eCDFs over d_{nearest} .

3.2 SDM Estimator over the Composed Input

Rather than calculating the SDM estimator over a single model, we take as input to g the hidden-states and output of the open-weight

model ($\mathcal{M}_a$) concatenated with the output from the closed-weight LMs ($\mathcal{M}_1$ and $\mathcal{M}_2$). Concretely,

$$\mathbf{h} = \left[\mathbf{h}_a, \hat{p}(y_a = 0 | \mathbf{x}_a), \hat{p}(y_a = 1 | \mathbf{x}_a), \right. \\ \left. \mathbf{1}_{\hat{y}_1=0} \cdot \hat{p}(y_1 | \mathbf{x}_1), \mathbf{1}_{\hat{y}_1=1} \cdot \hat{p}(y_1 | \mathbf{x}_1), \right. \\ \left. \mathbf{1}_{\hat{y}_2=0} \cdot \hat{p}(y_2 | \mathbf{x}_2), \mathbf{1}_{\hat{y}_2=1} \cdot \hat{p}(y_2 | \mathbf{x}_2) \right]^T \quad (5)$$

where the indicator functions, $\mathbf{1} \in \{0, 1\}$, in the final four terms construct soft one-hot vectors of the verbalized uncertainty corresponding to the prediction from each of $\mathcal{M}_1$ and $\mathcal{M}_2$, respectively.

4 Data: OpenVerification1

Training and calibrating the SDM estimator requires labeled data, $\mathcal{D}_{\text{tr}}$ and $\mathcal{D}_{\text{ca}}$. To facilitate constructing such estimators for programming and data science tasks, we release OPENVERIFICATION1, an Apache-2.0 licensed set of over two million examples from a selection of recent closed-weight LMs, as used for $\mathcal{M}_1$ and $\mathcal{M}_2$. The majority of the examples are from OpenThoughts2-1M [15], with negatives ($y = 0$) synthetically generated using GPT-4.1 (version 2025-04-14) [27].⁶ To construct task-specific agentic estimators, developers can supplement this data with samples drawn from the distribution of their chosen $\mathcal{M}_t$, $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_a$. We use a subset of OPENVERIFICATION1 for version 2.1.0 of the REEXPRESS MCP SERVER, described next, which serves as a demonstration of these capabilities with the publicly available OPENVERIFICATION1 data as the reference class for calibration, as well as a model that can be used out-of-the-box without training for use in low-risk settings.

5 System: Reexpress MCP Server

For version 2.1.0 of the REEXPRESS MCP SERVER, we construct the SDM estimator with OpenAI’s GPT-5.2 model (version 2025-12-11) [26] via the Microsoft Foundry service [24] as $\mathcal{M}_1$, and Google DeepMind’s Gemini-3-Pro-preview model [13, 14] as $\mathcal{M}_2$. We use the open-weight IBM Granite-3.3-8b-Instruct model [18] for $\mathcal{M}_a$. Each input to the SDM estimator (Eq. 5), $\mathbf{h} \in \mathbb{R}^{8198}$, is distilled into an exemplar vector, $\mathbf{h}' \in \mathbb{R}^{1000}$, by the 1-D CNN adaptor of the SDM activation. Exact L^2 matching is performed using the Faiss library [7, 19] with $|\mathcal{D}_{\text{tr}}| = 120, 158$. Compute costs are relatively modest for the device running the SERVER, since $\mathcal{M}_1$ and $\mathcal{M}_2$ are called via cloud-based APIs, and the modest-sized $\mathcal{M}_a$ only generates a single output token. As a general guideline, if the compute device can run $\mathcal{M}_a$, then the device can run the SERVER. This version of the SERVER is not calibrated to a specific tool-calling LM, $\mathcal{M}_t$, but the Server’s tool suite has been verified to work as intended with Anthropic’s Opus-4.6 [3].

Figure 2 provides a high-level overview of the system, and a detailed accounting of each tool is included in Appendix B. The SERVER takes as input the QUERY and RESPONSE mediated by $\mathcal{M}_t$,⁷ and returns $\hat{p}(y | \mathbf{x})$, the estimate of the predictive uncertainty over binary classification of instruction-following; whether the estimate falls within the HIGH-RELIABILITY region; and details of how the

⁵The earlier on-device macOS data analysis program, REEXPRESS ONE (https://github.com/ReexpressAI/Reexpress_one), released in November 2023, was a variation on this theme, with a visual user interface that facilitated calculating similar such thresholds directly over a standard 1-D CNN adaptor and softmax, taking the calibrated probability as a function of the frequency of points in the resulting class-wise partition.

⁶We aim to continue updating OPENVERIFICATION1 as new closed-weight models are released.

⁷We also provide a tool for sending the QUERY directly to $\mathcal{M}_1$ and $\mathcal{M}_2$, which is useful when the context is a large document or codebase, sidestepping the possibility of errors (and costs) from $\mathcal{M}_t$ in regenerating the context.

estimate was calculated, including the outputs from the closed-weight LMs, which $\mathcal{M}_t$ can use to refine its subsequent attempts to fulfill the user’s request. This process of refinement via tool-calling is, in effect, test-time search in the representation space of the SDM estimator, rather than the typical output “logit” space of the model.

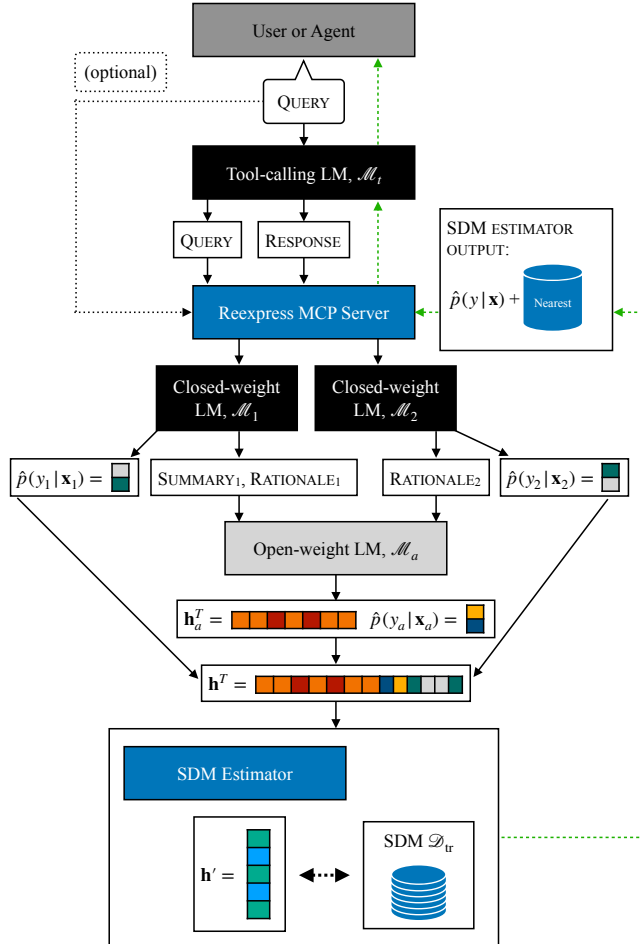


Figure 2: Overview of the Reexpress MCP Server, which provides $\hat{p}(y|x)$, an estimate of instruction-following of the tool-calling LM ($\mathcal{M}_t$), and interpretability-by-exemplar.

The SERVER enables a degree of dynamic *updatability* to closed-weight LMs as part of a search graph via tools to modify $\mathcal{D}_{tr}$ of the SDM estimator with additional examples labeled in $y \in \mathcal{Y}$, as well as a special out-of-distribution label, without retraining the estimator. The SDM estimator is a mix of parametric and non-parametric components; $\hat{y}$ stays constant with such changes to $\mathcal{D}_{tr}$, which serves as a persistent memory, but such changes do impact $\hat{p}(y|x)$. Thus, “fast-moving updates” by dynamically modifying $\mathcal{D}_{tr}$, and otherwise holding the parameters of the adaptor fixed, can impact the conditional branching decisions in a test-time search graph, with “slower-moving updates” achievable by running gradient descent on the parametric components of the adaptor layer when recalibrating

across all of the available data. In this way, the SERVER is a straightforward means of adding an adaptable, uncertainty-aware agentic memory to an existing LM pipeline, providing an additional avenue (beyond standard in-context adaptation) for continual learning to an otherwise static closed-weight LM.

NEAREST MATCH IN TRAINING

Successfully Verified (Prediction)

True

True Label

Verified

Model 1 Summary (gpt-5-2025-08-07)

The user asked to compute $2a - 3b + 4c$ for given vectors a, b, c and choose the correct option. The AI computed component-wise: $2a=(6,10,-2)$, $-3b=(-6,-6,-9)$, $4c=(16,-4,-12)$, summed to $(16, 0, -23)$, and selected option A.

Model 1 Explanation (gpt-5-2025-08-07)

The arithmetic is correct and fully shown; the resulting vector $(16, 0, -23)$ matches option A. The task is objective (not opinion-based) and does not depend on recent information. The answer relies solely on the user-provided context and basic vector operations.

Model 2 Explanation (gemini-2.5-pro)

The AI correctly performs the scalar multiplication and vector addition to arrive at the correct result. The step-by-step calculation is clear and accurate, matching the correct multiple-choice option.

Model 3 Agreement (granite-3.3-8b-instruct)

Do the model explanations agree that the response is correct?

Yes

Figure 3: The SDM estimator of the REEXPRESS MCP SERVER provides interpretability-by-exemplar by providing the nearest match to the prediction in $\mathcal{D}_{tr}$ of the SDM estimator.

In addition to the feedback to the tool-calling LM via the MCP protocol, the SERVER provides the output for the user to review via a locally saved HTML file (Figure 1). This includes the estimator details (Figure 5) and the nearest match from $\mathcal{D}_{tr}$ (Figure 3), which enables the user to *introspect* the estimate relative to the examples with known labels.

6 Evaluation

Appendix E provides a comprehensive analysis of the behavior of the SDM estimator compared to the verbalized uncertainty of the closed-weight LMs, including an ensemble thereof. We briefly summarize that analysis here.

Figure 4 is illustrative of the general behavior of the SDM estimator. The predictions tend to become more reliable with increasing $\text{SDM}(z')_{\hat{y}}$, the output of the SDM activation for the predicted class, and q' . This separation of the distribution enables the selective partitioning of Eq. 4.

Across sets, the SDM estimator remains well-calibrated. On the validation set of a well-studied multiple-choice set, which we hypothesize is in-distribution to $\mathcal{M}_1$ and $\mathcal{M}_2$, the verbalized uncertainty expressed in the output of the closed-weight models can itself be well-calibrated. However, as Table 1 and Table 2 show, the verbalized uncertainty, even when ensembled across models, is not sufficient for reliably calibrating the held-out set of OPENVERIFICATION1 constructed from OpenThoughts2-1M, especially as α exceeds the overall accuracy of $\mathcal{M}_1$ and $\mathcal{M}_2$.

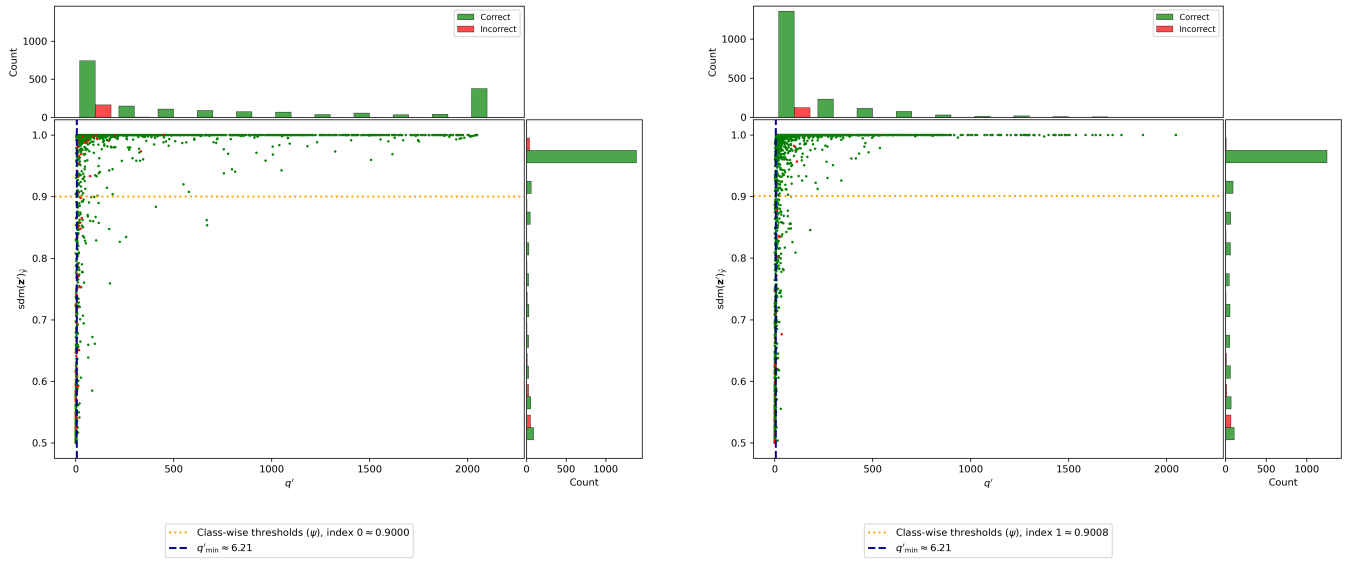


Figure 4: Classification results for class 0 (Not Verified), left, and class 1 (Verified), right, for the OPENTHOUGHTS-TEST-VERIFICATION evaluation data. The predictions become more reliable with increasing $\text{sdm}(z')_{\hat{y}}$, the output of the SDM activation for the predicted class, and q' . These graphs, the code for which is available in the REEXPRESS MCP SERVER repo, are interactive; clicking on a point prints (to standard out) detailed information on a document and the estimate of the predictive uncertainty.

7 Guidance

The REEXPRESS MCP SERVER is released with a pre-trained SDM estimator that can be used without additional training for low-risk programming and data science tasks, calibrated with $\alpha = 0.9$ against OPENVERIFICATION1. For production settings, and other-wise domain-specific settings, we recommend calibrating directly over samples from the distribution of M_t , M_1 , M_2 , and M_a , as used in the particular target agentic pipeline. A discussion of limitations is included in Appendix A.

8 Conclusion

The REEXPRESS MCP SERVER reflects the evolution of, and lessons learned from, explainable and interpretable AI over the last decade, from early endogenous modeling of explanations with LMs as classifiers [34], to actionable notions of interpretability by analyzing neural networks as hidden instance-based metric learners [31], and finally, to robust, interpretable estimators of the predictive uncertainty [32]. Closed-weight LMs are a weaker substrate for these modeling innovations than open-weight LMs, but we can substantively improve the robustness, adaptability, and interpretability of closed-weight tool-calling LMs via composition with multiple models as input to an SDM estimator calibrated against samples from the target reference class for a given multi-stage agentic system. The SERVER, training code, and OPENVERIFICATION1 dataset simplify the construction of such domain-specific estimators.

Acknowledgments

This project was supported by Reexpress AI, Inc. We thank the Google for Startups Cloud Program and the Microsoft for Startups

program for compute support and access to state-of-the-art closed-weight LMs.

References

- [1] Anastasios Nikolas Angelopoulos, Stephen Bates, Michael Jordan, and Jitendra Malik. 2021. Uncertainty Sets for Image Classifiers using Conformal Prediction. In *International Conference on Learning Representations*. https://openreview.net/forum?id=eNdiU_DbM9
- [2] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Accessed: 2026-03-09.
- [3] Anthropic. 2026. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>. Closed-weight language model (claude-opus-4-6) product announcement.
- [4] Glenn W. Brier. 1950. Verification Of Forecasts Expressed In Terms Of Probability. *Monthly Weather Review* 78, 1 (1950), 1–3. doi:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2
- [5] C. K. Chow. 1957. An optimum character recognition system using decision functions. *IRE Transactions on Electronic Computers* EC-6, 4 (1957), 247–254. doi:10.1109/TEC.1957.5222035
- [6] A. P. Dawid. 1982. The Well-Calibrated Bayesian. *J. Amer. Statist. Assoc.* 77, 379 (1982), 605–610. doi:10.1080/01621459.1982.10477856
- [7] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. THE FAISS LIBRARY. *IEEE Transactions on Big Data* (2025), 1–17. doi:10.1109/TBDATA.2025.3618474
- [8] A. Dvoretzky, J. Kiefer, and J. Wolfowitz. 1956. Asymptotic Minimax Character of the Sample Distribution Function and of the Classical Multinomial Estimator. *The Annals of Mathematical Statistics* 27, 3 (1956), 642–669. doi:10.1214/aoms/1177728174
- [9] Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. In *Proceedings of The 33rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 48)*, Maria Florina Balcan and Kilian Q. Weinberger (Eds.). PMLR, New York, New York, USA, 1050–1059. <https://proceedings.mlr.press/v48/gal16.html>
- [10] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/4a8423d5e91fda00bb7e46540e2b0cf1-Paper.pdf

- [11] Gemini Team, Google. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs.CL] <https://arxiv.org/abs/2507.06261>
- [12] Google DeepMind. 2025. Gemini 2.5: Our most intelligent AI model. <https://blog.google/innovation-and-ai/models-and-research/google-deepmind/gemini-model-thinking-updates-march-2025/>. Closed-weight language model (gemini-2.5-pro) product announcement.
- [13] Google DeepMind. 2025. *Gemini 3 Pro Model Card*. Technical Report. Google DeepMind. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>
- [14] Google DeepMind. 2025. A new era of intelligence with Gemini 3. <https://blog.google/products/gemini/gemini-3/>. Closed-weight language model (gemini-3-pro-preview) product announcement.
- [15] Etash Kumar Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Rea Sprague, Ashima Suvarna, Benjamin Feuer, Leon Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah M Pratt, Vivek Ramanujan, Jon Saad-Falcon, Stutee Acharya, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alex Dimakis, and Ludwig Schmidt. 2026. OpenThoughts: Data Recipes for Reasoning Models. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=7xjoTuaNmN>
- [16] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1321–1330. <https://proceedings.mlr.press/v70/guo17a.html>
- [17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=d7KBjml3GmQ>
- [18] IBM Granite Team. 2025. ibm-granite/granite-3.3-8b-instruct. <https://huggingface.co/ibm-granite/granite-3.3-8b-instruct> Open-weight language model.
- [19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [20] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1412.6980>
- [21] Meelis Kull, Miquel Perello-Nieto, Markus Kängsepp, Telmo Silva Filho, Hao Song, and Peter Flach. 2019. Beyond temperature scaling: Obtaining well-calibrated multiclass probabilities with Dirichlet calibration. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/8ca01ea920679a0fe3728441494041b9-Paper.pdf
- [22] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/9ef2ed4b7fd2c810847ffa5fa85bce38-Paper.pdf
- [23] Microsoft. 2025. GPT-5 in Azure AI Foundry: The future of AI apps and agents starts here. <https://azure.microsoft.com/en-us/blog/gpt-5-in-azure-ai-foundry-the-future-of-ai-apps-and-agents-starts-here/>. Closed-weight language model (gpt-5-2025-08-07) product announcement. OpenAI GPT-5 accessed via the Microsoft Azure AI Foundry service.
- [24] Microsoft. 2025. Introducing GPT-5.2 in Microsoft Foundry: The new standard for enterprise AI. <https://azure.microsoft.com/en-us/blog/introducing-gpt-5-2-in-microsoft-foundry-the-new-standard-for-enterprise-ai/>. Closed-weight language model (gpt-5.2-2025-12-11) product announcement. OpenAI GPT-5.2 accessed via the Microsoft Foundry service.
- [25] OpenAI. 2025. GPT-5 is here. <https://openai.com/gpt-5/>. Closed-weight language model (gpt-5-2025-08-07) product announcement.
- [26] OpenAI. 2025. Introducing GPT-5.2. <https://openai.com/index/introducing-gpt-5-2/>. Closed-weight language model (gpt-5.2-2025-12-11) product announcement.
- [27] OpenAI. 2025. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/>. Closed-weight language model (gpt-4.1-2025-04-14) product announcement.
- [28] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. 2019. Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/8558cb408c1d76621371888657d2eb1d-Paper.pdf
- [29] Melissa Z. Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, Shu Liu, Tianneng Shi, Xiaoyuan Liu, Jared Quincy Davis, Emmanuele Lacavalla, Alessandro Basile, Shuyi Yang, Paul Castro, Daniel Kang, Joseph E. Gonzalez, Koushik Sen, Dawn Song, Ion Stoica, Matei Zaharia, and Marquita Ellis. 2026. Measuring Agents in Production. arXiv:2512.04123 [cs.CY] <https://arxiv.org/abs/2512.04123>
- [30] John C. Platt. 1999. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*. MIT Press, 61–74.
- [31] Allen Schmaltz. 2021. Detecting Local Insights from Global Labels: Supervised and Zero-Shot Sequence Labeling via a Convolutional Decomposition. *Computational Linguistics* 47, 4 (Dec. 2021), 729–773. doi:10.1162/coli_a_00416
- [32] Allen Schmaltz. 2025. Similarity-Distance-Magnitude Activations. arXiv:2509.12760 [cs.LG] <https://arxiv.org/abs/2509.12760> To appear in *Findings of the Association for Computational Linguistics: ACL 2026*, San Diego, CA, USA.
- [33] Allen Schmaltz, Yoon Kim, Alexander Rush, and Stuart Shieber. 2017. Adapting Sequence Models for Sentence Correction. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, Martha Palmer, Rebecca Hwa, and Sebastian Riedel (Eds.). Association for Computational Linguistics, Copenhagen, Denmark, 2807–2813. doi:10.18653/v1/D17-1298
- [34] Allen Schmaltz, Yoon Kim, Alexander M. Rush, and Stuart Shieber. 2016. Sentence-Level Grammatical Error Identification as Sequence-to-Sequence Correction. In *Proceedings of the 11th Workshop on Innovative Use of NLP for Building Educational Applications*, Joel Tetreault, Jill Burstein, Claudia Leacock, and Helen Yannakoudakis (Eds.). Association for Computational Linguistics, San Diego, CA, 242–251. doi:10.18653/v1/W16-0528
- [35] Juozas Vaicenavicius, David Widmann, Carl Andersson, Fredrik Lindsten, Jacob Roll, and Thomas Schön. 2019. Evaluating model calibration in classification. In *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 89)*, Kamalika Chaudhuri and Masashi Sugiyama (Eds.). PMLR, 3459–3467. <https://proceedings.mlr.press/v89/vaicenavicius19a.html>
- [36] Vladimir Vovk, Alex Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer-Verlag, Berlin, Heidelberg.

A Limitations

With access to the hidden-states of $\mathcal{M}_l$, we could directly construct an SDM estimator for classification of instruction-following that would exhibit the covariate shift robustness established in existing work [32]. Here, with access only to the generated output tokens of a closed-weight $\mathcal{M}_l$, we instead construct an approximation of the desired quantity via an ensemble of $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_a$. $\mathcal{M}_1$ and $\mathcal{M}_2$ are themselves closed-weight LMs, the generated output of which itself is not expected to be particularly robust to distribution shifts since it is likely the result of standard output softmax layers, rather than constraints over the representation space of the support set of the model. Further, to keep the context length sufficiently short to enable on-device processing, $\mathcal{M}_a$ operates at the level of abstraction of agreement between $\mathcal{M}_1$ and $\mathcal{M}_2$. These compromises reflect the compute and resource constraints of typical on-device developer settings with contemporary LMs, but they diminish the robustness of the SDM estimator to distribution shifts.

Given these inherent limitations, the most important recommendation for production agentic applications is to calibrate against samples from the target pipeline. For example, if the agentic pipeline is intended to close support tickets for a particular system, examples of successes and failures can be collected and those can serve as the calibration set. The REEXPRESS MCP SERVER repo provides code for training an SDM estimator for such domain-specific settings.

With additional compute, one straightforward (from a modeling perspective) variation would be to add an additional, larger open-weight model, $\mathcal{M}_3$, which would operate similarly to $\mathcal{M}_2$, but

the hidden-states of which would contribute to $\mathbf{h}$ (Eq. 5). Calibrating with such an $\mathcal{M}_3$ would incur an additional one-time cost to train the SDM estimator, as well as increased test-time costs. The increased test-time costs may necessitate a more complex, non-local deployment of the SERVER. We leave a controlled study of the most productive use of substantively more compute to future research.

B MCP Tools provided by the Reexpress MCP Server

The following nine MCP tools are available in version 2.1.0 of the REEXPRESS MCP SERVER. Additional details are provided in the documentation in the code repo. In the following, $\emptyset$ indicates an absence of a required input argument and/or side-effect. We start with the *Main tool*, *Reexpress*, and categorize the remaining tools as *Auxiliary helper tools*, *Model update tools*, and *File Access tools*.

- *Main tool*
 - *Reexpress*
Input: QUERY and RESPONSE
Side-effect: The cache is cleared of any existing estimates, and the new estimate is added to the cache. Optionally, HTML output is saved to disk to a directory specified in the SERVER config.
Output: Output from the SDM estimator
- *Auxiliary helper tools*
 - *ReexpressView*
Input: $\emptyset$
Side-effect: $\emptyset$
Output: Output from the SDM estimator if an estimate exists in the cache, else failure message
 - *ReexpressReset*
Input: $\emptyset$
Side-effect: The sequential tool-call limit counter is reset.⁸
Output: Success/failure message
- *Model update tools*
 - *ReexpressAddFalse*
Input: $\emptyset$
Side-effect: If an estimate exists in the cache, it is added to $\mathcal{D}_{tr}$ with $y = 0$. The cache is cleared of any existing estimate.
Output: Success/failure message
 - *ReexpressAddTrue*
Input: $\emptyset$
Side-effect: If an estimate exists in the cache, it is added to $\mathcal{D}_{tr}$ with $y = 1$. The cache is cleared of any existing estimate.
Output: Success/failure message
 - *ReexpressAddOOD*
Input: $\emptyset$
Side-effect: If an estimate exists in the cache, it is added to $\mathcal{D}_{tr}$ with $y = \perp$. The cache is cleared of any existing estimate.

⁸The SERVER config has user-settable soft and hard limits for the total number of times the main Reexpress tool can be called. The soft limit can be reset with the ReexpressReset tool, whereas the hard limit requires a server restart. These limits guard against unexpected, runaway calls by $\mathcal{M}_t$ to the SERVER, and by extension, $\mathcal{M}_1$ and $\mathcal{M}_2$.

- *File Access tools* (disabled by default in the SERVER config)
 - *ReexpressDirectorySet*
Input: Directory string
Side-effect: If the directory string is permitted by the SERVER config, it is appended to the set of parent directories. (No file contents are read.)
Output: Success/failure message
 - *ReexpressFileSet*
Input: Filename, or absolute path with filename
Side-effect: If the file exists and is allowed, its contents are read and appended to a local file cache, which will be sent directly to $\mathcal{M}_1$ and $\mathcal{M}_2$ the next time the Reexpress tool is called.
Output: Success/failure message
 - *ReexpressFileClear*
Input: $\emptyset$
Side-effect: The file cache is cleared.
Output: Success/failure message

C Additional Details

The first version of the REEXPRESS MCP SERVER was released on May 9, 2025. We describe here specifically version 2.1.0, which was released on January 12, 2026.

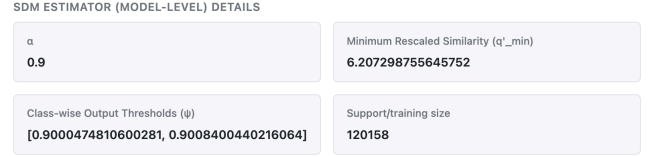


Figure 5: SDM estimator details for version 2.1.0 of the Reexpress MCP Server.

C.1 SDM Estimator training details

The training scripts for the SDM estimator in version 2.1.0 of the SERVER are provided in the public release code and can be used as a guide for constructing domain-specific estimators. We set $M = 1000$ and use a mini-batch size of 50. $\mathbf{h}$ is mean-centered using summary statistics over $\mathcal{D}_{tr}$. We use the Adam optimizer [20] (without weight decay) with a learning rate of 5×10^{-5} . We train for 5 iterations of random shuffles of $\mathcal{D}_{tr}$ and $\mathcal{D}_{ca}$, each of 500 epochs, choosing the model among all 2,500 epochs with the lowest balanced (across classes) SDM loss over $\mathcal{D}_{ca}$.

The full OPENVERIFICATION1 dataset consists of output from multiple state-of-the-art closed-weight LMs as they were released over the course of 2025. Generating from the models is relatively expensive at scale. To keep API costs manageable, $\mathcal{D}_{tr}$ and $\mathcal{D}_{ca}$ of the SDM estimator released in version 2.1.0 of the SERVER supplemented the samples from GPT-5.2 and Gemini-3-Pro with additional samples from the previous versions of those model lines. Specifically, we used the existing samples in OPENVERIFICATION1 for OpenAI’s GPT-5 model (version 2025-08-07) [25] collected via the Microsoft Azure AI Foundry service [23] as additional samples for $\mathcal{M}_1$, and

we used Google DeepMind’s Gemini-2.5-Pro model [11, 12] as additional samples for $\mathcal{M}_2$. Generally speaking this type of mixing should be approached with caution, but in this case, analysis of calibration on held-out sets for the target $\mathcal{M}_1$ and $\mathcal{M}_2$ verified the expected behavior and justified the cost-savings.

D Prompts

D.1 Prompt for $\mathcal{M}_1$

The “developer” role for $\mathcal{M}_1$ is as follows:

You are a helpful assistant that verifies instruction following. You classify whether or not the provided AI response correctly and faithfully answers the user’s questions or instructions. The user’s questions or instructions are contained within the XML tags `<question>` and `</question>`, and the LLM outputs are contained within the XML tags `<ai_response>` and `</ai_response>`. Additionally, you indicate whether the output is open to opinion, or otherwise subjective, as well as whether the answer may be dependent on more recent information than you currently have access to. The user’s instruction may not always be answerable with the provided context and may not be answerable via your internal knowledge, so to help the user understand the reliability of your response, indicate whether your response is based on the context provided by the user, your internal knowledge, or some combination thereof. Think carefully and systematically about each step of your reasoning process. For science, math, programming, and engineering problems, consider multiple approaches to solve the problem and whether those approaches arrive at the same solution. If you are uncertain of your response, please err on the side of predicting the verification is false, rather than providing a classification that you cannot clearly justify based on the context and/or your internal knowledge. Please structure your response using the provided JSON format. Provide a brief summary of the question and AI response. Please provide your binary classification verification where True indicates that you can verify that the response answered the query or instruction, and False indicates you cannot verify that the response answered the query or instruction. Provide a confidence estimate in your verification classification as a probability between 0 and 1, where a probability of 0 indicates no confidence and a probability of 1 indicates 100% confidence in your predicted classification. Finally, please provide a short explanation for your confidence in your classification. If you

are unable to provide a True verification classification with a probability of at least 0.95, your short explanation should also briefly indicate what additional information (such as additional references, documentation, or tool output) from the user might be able to improve your confidence in your classification.

The “user” role for $\mathcal{M}_1$ is as follows:

`<question> AQ A*</question> <ai_response> R </ai_response>`

where A is replaced with any optional content from the file set tools (Appendix B) within `<attached_file></attached_file>` tags to bypass mediation from $\mathcal{M}_t$. If the file content is not empty, A*, is replaced with

(Note: I have included additional documents relevant to this discussion within the `<attached_file></attached_file>` XML tags.).

If the file content is empty, A and A* are empty strings. Q and R are replaced with the `QUERY` and `RESPONSE`, respectively, provided by $\mathcal{M}_t$ as arguments to the Reexpress tool (Appendix B).

D.2 Prompt for $\mathcal{M}_2$

The “system instruction” for $\mathcal{M}_2$ is as follows:

You are a helpful assistant that verifies instruction following. Given one or more user instructions and LLM output(s), you classify whether or not the output correctly and faithfully answers the user’s question(s) and/or instruction(s). The user’s question(s) and/or instruction(s) are contained within the XML tags `<question>` and `</question>`, and the LLM output(s) are contained within the XML tags `<ai_response>` and `</ai_response>`. Additionally, you indicate whether the output is open to opinion, or otherwise subjective, as well as whether the answer may be dependent on more recent information than you currently have access to. The user’s instruction may not always be answerable with the provided context and may not be answerable via your internal knowledge, so to help the user understand the reliability of your response, indicate whether your response is based on the context provided by the user, your internal knowledge, or some combination thereof. If you are uncertain of your response, please err on the side of predicting the verification is false, rather than providing a classification that you cannot clearly justify based on the context and/or your internal knowledge. Please structure your response using the provided JSON format. Please provide your binary classification verification where true indicates that you can verify that the response answered the query or instruction, and false indicates you cannot verify that the response

answered the query or instruction. Provide a confidence estimate in your verification classification as a probability between 0 and 1, where a probability of 0 indicates no confidence and a probability of 1 indicates 100% confidence in your predicted classification. Finally, please provide a short explanation for your confidence in your classification. If you are unable to provide a True verification classification with a probability of at least 95%, your short explanation should also briefly indicate what additional information (such as additional references, documentation, or tool output) from the user might be able to improve your confidence in your classification.

The user query content for M_2 is the same as the “user” role for M_1 described above.

D.3 Prompt for M_a

```
<topic> S1 </topic> Do the following model
explanations agree that the response is correct?
<model1_explanation> R1 </model1_explanation>
<model2_explanation> R2 </model2_explanation>
Yes or No?
```

where S1, R1, and R2 are replaced by SUMMARY₁, RATIONALE₁, and RATIONALE₂, respectively.

D.4 Recommended tool-calling prompt for M_t

With Anthropic’s Opus-4.6 model as the tool-calling LM, M_t , the Server’s main tool has been verified to work as intended with the following prompt.

```
Please verify your final answer with the
Reexpress tool. Do not include the Reexpress
instructions themselves as part of the user_question
argument of the Reexpress tool. Structure
your input to the ai_response argument as
Reference [Number, source or "internal knowledge"
or "internal reasoning"; and if applicable,
URL or filename, and the doc_index-sentence_index]:
Source text; Reference [Number+1, source or
"internal knowledge" or "internal reasoning";
and if applicable, URL or filename, and
the doc_index-sentence_index]: Source text;
Answer: Your answer. Consider your final
answer verified if <successfully_verified>
True </successfully_verified> and <confidence>
>= 90% </confidence>.
```

The above can serve as a baseline starting point; variations on this theme include instructing the model to continue the verification loop (with access to additional tools, such as a coding terminal or web search) for a particular maximum number of iterations until the 90% confidence level is achieved.

We provide example prompts for each of the other tools in the code repo.

E Evaluation

E.1 Data

We consider two splits of OPENVERIFICATION1 for evaluation, among examples with output from both GPT-5.2 and Gemini-3-Pro-preview. The first is the validation split of MMLU [17] (MMLU-VALIDATION-VERIFICATION), which consists of multiple choice questions. The positive examples (i.e., those with $y = 1$) use the correct answer letter as the RESPONSE from M_t , and the negative examples (i.e., those with $y = 0$) are constructed by randomly choosing a wrong answer letter. The split consists of 2,526 examples. It is assumed that these questions, or demonstrably similar variations, likely appear in the training and/or post-training sets of M_1 and M_2 since MMLU is a commonly used benchmark dataset and these examples are from the validation split; however, they do not occur directly in $\mathcal{D}_{tr}$ nor $\mathcal{D}_{ca}$ of the SDM estimator. We also consider a held-out set of examples from the OpenThoughts2-1M subset of OPENVERIFICATION1, with negatives ($y = 0$) synthetically generated using GPT-4.1 (version 2025-04-14). This split consists of 3,956 examples, for which we use the label OPENTHOUGHTS-TEST-VERIFICATION.

E.2 Models and Estimators

We consider the SDM estimator of version 2.1.0 of the SERVER (REEXPRESSV2.1.0) using the selection criteria of Eq. 4 at α with the label SDM_{HR} . The label SDM_{α} represents a simple thresholding of the output of the SDM activation (Eq. 1) at α . For reference, we also threshold the SDM activation at α with $q = e - 2$, using the label $SOFTMAX(d \cdot z')$, to illustrate the impact of q , and we threshold the SDM activation at α with $q = e - 2$ and $d = 1$, which is equivalent to a standard SOFTMAX over the adaptor layer, using the label SOFTMAX.

For comparison, we consider $\hat{p}(y_1 | x_1)$, the verbalized probability from GPT-5.2 thresholded at α (VERBALIZEDPROB), and $\hat{p}(y_2 | x_2)$, the verbalized probability from GEMINI-3 thresholded at α (VERBALIZEDPROB). Since the SDM estimator in effect ensembles both of these estimators (via h), we also compare to a direct ensemble of these models (GPT-5.2+GEMINI-3). The selection criteria of the VERBALIZEDPROB estimator of the ensemble is calculated as $(\hat{y}_1 = \hat{y}_2) \wedge \hat{p}(y_1 | x_1) \geq \alpha \wedge \hat{p}(y_2 | x_2) \geq \alpha$ (i.e., the predicted labels match for both models *and* the verbalized probability is at least α for both models).

For reference, the label NO-REJECT refers to predictions without any selective filtering. In the case of the ensemble model (GPT-5.2+GEMINI-3), AGREE-OR-REJECT refers to the set of examples for which $\hat{y}_1 = \hat{y}_2$ (i.e., the predicted labels match for both models).

We consider $\alpha = 0.9$, as in the released model of the SERVER. For reference, we also recalibrate the estimator at the more stringent $\alpha = 0.95$.

E.3 Results

Figure 4 is illustrative of the general behavior of the SDM estimator. The predictions tend to become more reliable with increasing $SDM(z')_{\hat{y}}$, the output of the SDM activation for the predicted class, and q' . This separation enables the selective partitioning of Eq. 4.

Table 1 provides the main results for $\alpha = 0.9$. On the MMLU-VALIDATION-VERIFICATION data, which is assumed to be a

close in-distribution dataset to $\mathcal{M}_1$ and $\mathcal{M}_2$, even the `VERBALIZEDPROB` estimators from the individual models are well-calibrated. However, for the `OPENTHOUGHTS-TEST-VERIFICATION` data, some of the conditional estimates using the verbalized probability fall below the desired α , while the SDM estimator remains well-calibrated.

Table 2 contains the results for the more stringent $\alpha = 0.95$, for which the SDM estimator's $q'_{\min} \approx 39.01$, as opposed to $q'_{\min} \approx 6.21$ when calibrated at $\alpha = 0.9$. In this case, the SDM estimator remains well-calibrated while the marginal and most of the conditional

estimates based directly on the verbalized uncertainty fall below $\alpha = 0.95$.

One observation particular to release 2.1.0 of the `SERVER` is that with GPT-5.2 as $\mathcal{M}_1$ and Gemini-3-Pro-preview model as $\mathcal{M}_2$, the examples in `OPENVERIFICATION1` are now on the “easy side” for these models, relative to earlier versions of the `SERVER` that used previous OpenAI and Google Deepmind models for the closed-weight models. In our continuing expansion of `OPENVERIFICATION1`, we aim to add increasingly challenging examples in subsequent releases.

Table 1: Comparison of estimators, $\alpha = 0.9$. R indicates all predictions were rejected, which is preferred over falling under the expected accuracy. $n = |\text{Admitted}|$, the count of non-rejected documents.

Dataset	Model	Estimator	Class-conditional		Prediction-conditional		Marginal	
			$y = 0$	$y = 1$	$\hat{y} = 0$	$\hat{y} = 1$	$y \in \{0, 1\}$	
			Acc.	$\frac{n}{ \mathcal{D}_{\text{re}} }$	Acc.	$\frac{n}{ \mathcal{D}_{\text{re}} }$	Acc.	$\frac{n}{ \mathcal{D}_{\text{re}} }$
MMLU-Validation-Verification	GPT-5.2	NO-REJECT	0.99	0.49	0.89	0.51	0.90	0.54
MMLU-Validation-Verification	GPT-5.2	VERBALIZEDPROB	0.99	0.45	0.96	0.46	0.96	0.46
MMLU-Validation-Verification	GEMINI-3	NO-REJECT	0.97	0.49	0.96	0.51	0.96	0.49
MMLU-Validation-Verification	GEMINI-3	VERBALIZEDPROB	0.97	0.49	0.96	0.51	0.96	0.49
MMLU-Validation-Verification	GPT-5.2+GEMINI-3	AGREE-OR-REJECT	0.99	0.48	0.96	0.47	0.96	0.49
MMLU-Validation-Verification	GPT-5.2+GEMINI-3	VERBALIZEDPROB	0.99	0.44	0.97	0.45	0.97	0.45
MMLU-Validation-Verification	REEXPRESSV2.1.0	NO-REJECT	0.97	0.49	0.88	0.51	0.88	0.53
MMLU-Validation-Verification	REEXPRESSV2.1.0	SOFTMAX	0.98	0.46	0.92	0.43	0.93	0.49
MMLU-Validation-Verification	REEXPRESSV2.1.0	SOFTMAX($d \cdot z'$)	0.99	0.16	0.98	0.12	0.98	0.16
MMLU-Validation-Verification	REEXPRESSV2.1.0	SDM $_{\alpha}$	0.99	0.36	0.96	0.33	0.96	0.37
MMLU-Validation-Verification	REEXPRESSV2.1.0	SDM $_{\text{HR}}$	0.99	0.36	0.96	0.33	0.96	0.37
OPENTHOUGHTS-Test-Verification	GPT-5.2	NO-REJECT	0.94	0.50	0.80	0.50	0.82	0.57
OPENTHOUGHTS-Test-Verification	GPT-5.2	VERBALIZEDPROB	0.95	0.48	0.87	0.46	0.88	0.52
OPENTHOUGHTS-Test-Verification	GEMINI-3	NO-REJECT	0.92	0.50	0.89	0.50	0.89	0.51
OPENTHOUGHTS-Test-Verification	GEMINI-3	VERBALIZEDPROB	0.92	0.50	0.89	0.50	0.89	0.51
OPENTHOUGHTS-Test-Verification	GPT-5.2+GEMINI-3	AGREE-OR-REJECT	0.95	0.48	0.88	0.46	0.89	0.51
OPENTHOUGHTS-Test-Verification	GPT-5.2+GEMINI-3	VERBALIZEDPROB	0.95	0.47	0.89	0.45	0.90	0.50
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	NO-REJECT	0.91	0.50	0.94	0.50	0.93	0.49
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SOFTMAX	0.95	0.45	0.97	0.42	0.97	0.44
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SOFTMAX($d \cdot z'$)	0.99	0.23	1.00	0.16	1.00	0.23
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SDM $_{\alpha}$	0.97	0.38	0.99	0.34	0.99	0.37
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SDM $_{\text{HR}}$	0.97	0.37	0.99	0.33	0.99	0.36

Table 2: Comparison of estimators, $\alpha = 0.95$. R indicates all predictions were rejected, which is preferred over falling under the expected accuracy. $n = |\text{Admitted}|$, the count of non-rejected documents.

Dataset	Model	Estimator	Class-conditional		Prediction-conditional		Marginal	
			$y = 0$	$y = 1$	$\hat{y} = 0$	$\hat{y} = 1$	$y \in \{0, 1\}$	
			Acc.	$\frac{n}{ \mathcal{D}_{\text{re}} }$	Acc.	$\frac{n}{ \mathcal{D}_{\text{re}} }$	Acc.	$\frac{n}{ \mathcal{D}_{\text{re}} }$
MMLU-Validation-Verification	GPT-5.2	NO-REJECT	0.99	0.49	0.89	0.51	0.90	0.54
MMLU-Validation-Verification	GPT-5.2	VERBALIZEDPROB	0.99	0.44	0.97	0.45	0.97	0.44
MMLU-Validation-Verification	GEMINI-3	NO-REJECT	0.97	0.49	0.96	0.51	0.96	0.49
MMLU-Validation-Verification	GEMINI-3	VERBALIZEDPROB	0.97	0.48	0.96	0.50	0.96	0.48
MMLU-Validation-Verification	GPT-5.2+GEMINI-3	AGREE-OR-REJECT	0.99	0.48	0.96	0.47	0.96	0.49
MMLU-Validation-Verification	GPT-5.2+GEMINI-3	VERBALIZEDPROB	0.99	0.43	0.98	0.44	0.98	0.43
MMLU-Validation-Verification	REEXPRESSV2.1.0	NO-REJECT	0.97	0.49	0.88	0.51	0.88	0.53
MMLU-Validation-Verification	REEXPRESSV2.1.0	SOFTMAX	0.98	0.45	0.93	0.39	0.94	0.47
MMLU-Validation-Verification	REEXPRESSV2.1.0	SOFTMAX($d \cdot z'$)	0.99	0.11	0.98	0.08	0.99	0.11
MMLU-Validation-Verification	REEXPRESSV2.1.0	SDM $_{\alpha}$	0.99	0.34	0.96	0.31	0.97	0.35
MMLU-Validation-Verification	REEXPRESSV2.1.0	SDM $_{\text{HR}}$	0.99	0.33	0.96	0.29	0.96	0.34
OPENTHOUGHTS-Test-Verification	GPT-5.2	NO-REJECT	0.94	0.50	0.80	0.50	0.82	0.57
OPENTHOUGHTS-Test-Verification	GPT-5.2	VERBALIZEDPROB	0.95	0.48	0.88	0.44	0.89	0.50
OPENTHOUGHTS-Test-Verification	GEMINI-3	NO-REJECT	0.92	0.50	0.89	0.50	0.89	0.51
OPENTHOUGHTS-Test-Verification	GEMINI-3	VERBALIZEDPROB	0.92	0.49	0.89	0.50	0.89	0.51
OPENTHOUGHTS-Test-Verification	GPT-5.2+GEMINI-3	AGREE-OR-REJECT	0.95	0.48	0.88	0.46	0.89	0.51
OPENTHOUGHTS-Test-Verification	GPT-5.2+GEMINI-3	VERBALIZEDPROB	0.95	0.47	0.90	0.43	0.91	0.49
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	NO-REJECT	0.91	0.50	0.94	0.50	0.93	0.49
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SOFTMAX	0.96	0.44	0.97	0.37	0.98	0.43
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SOFTMAX($d \cdot z'$)	0.99	0.18	1.	0.10	1.	0.18
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SDM $_{\alpha}$	0.97	0.36	0.99	0.32	0.99	0.35
OPENTHOUGHTS-Test-Verification	REEXPRESSV2.1.0	SDM $_{\text{HR}}$	0.98	0.33	0.99	0.25	0.99	0.33